

The Longest Sorted Bucket Sequence: An Algorithmic Bridge Between LCS and 2D LIS

Awwal Mohammed¹, Caroline S. Selvarajah²

² Faculty of Arts and Science, Universiti Malaya-Wales, Malaysia

¹xxxxx@tuta.io; ²xxxxxx@umwales.edu.my

ABSTRACT

This paper introduces the Longest Sorted Bucket Sequence (LSBS) problem, an algorithmic conceptual framework that extends classical subsequence analysis by incorporating ordered, non-reusable bucket-based constraints. We provide a formal definition of LSBS and demonstrate its fundamental connection to the Longest Common Subsequence (LCS) problem, showing that any LCS instance can be efficiently transformed into an LSBS instance. We then develop two distinct algorithms that solves LSBS: a straightforward $O(M^2)$ time sparse dynamic programming approach; and a more sophisticated $O(M \log M)$ time algorithm, where M is the size of the matching list (or the second row of what we call the Common Element Table (CET)). The efficient algorithm is achieved by employing a two-dimensional (2D) sort which in essence embodies the point domination problem, followed by a typical one-dimensional (1D) Longest Increasing Subsequence (LIS) search, a technique analogous to Patience Sorting. We show that this methodology effectively rediscovers the principles of the renowned Hunt-Szymanski algorithm for LCS, offering a simplified and intuitive perspective on its mechanics, while bounding auxiliary space complexity to $O(|M|)$ in a fashion inspired by but not as strict as Hirschberg's widely acclaimed linear space LCS solution. The total time complexity for solving LCS using this LSBS strategy is $O(|A| + |B| + M \log M)$ where $|A|$ and $|B|$ are the lengths of the input sequences. Our analysis is supported by implementations and decussated with discussion of trade-offs with other known LCS algorithms, validating the correctness and performance of the proposed framework.

Keywords: Longest Common Subsequence, Longest Increasing Subsequence, Sorting, Dynamic Programming, 2D Point Dominance

I. INTRODUCTION

The study of sequences and subsequences is a widely explored undertaking in computer science, with profound applications in bioinformatics, data compression, text analysis, and beyond. The Longest Common Subsequence (LCS) and Longest Increasing Subsequence (LIS) problems are canonical examples, representing fundamental challenges that have been studied for decades, with the former being notably published by Wagner & Fischer [1] and the latter, Erdős & Szekeres [2]. While elegant solutions for these problems are well-established, the exploration of related variants continues to yield theoretical insights and practical tools.

In this paper, we introduce a conceptual framework called the Longest Sorted Bucket Sequence (LSBS) problem. LSBS involves selecting a sequence of numbers from an ordered collection of “buckets”, each of which contain distinctly sorted numbers, subject to the constraint that each chosen number must be greater than the previous (i.e. strictly increasing), and each must originate from a later bucket than the previous. This problem formalizes a structure inherent in many real-world scenarios such as identifying monotonically increasing trends in time-series data grouped by sampling intervals [3], or scheduling tasks that are batched by priority level [4].

The primary contribution of this work is twofold. First, we establish a formal reduction, demonstrating that the classical LCS problem can be transformed into an instance of LSBS. This is achieved by constructing a Common Element Table (CET) data structure, where for each character in the first sequence A , we create a bucket containing the indices of its occurrences in the second sequence B . An LCS of A and B then corresponds directly to an LSBS of this bucket collection.

Second, we explore algorithms to solve the LSBS problem itself, beginning with a baseline dynamic programming solution and then develop an efficient $O(M \log M)$ algorithm, where M is the total number of elements across all buckets. This algorithm lays its foundational momentum on a specialized “bilateral sort” technique followed by a one-dimensional LIS computation, a process that is functionally equivalent to the Patience Sorting algorithm [5]. In doing so, we demonstrate that our LSBS-centric approach to solving LCS provides a new and powerful lens for understanding the Hunt-Szymanski algorithm [6] (one of the most efficient methods for finding LCS) while distilling the complicated aspects using easily understood data structures and algorithmic modelling techniques.

Given the advent and popularity of LCS and LIS in computer science discourse and algorithms literature, we have pragmatically focused on delivering our core algorithms and the processes around them, while deprioritizing verbosity of literature review because these topics have been extensively and rigorously covered by countless high-profile researchers [7, 8, 9, 10]. We instead provide an alternatively effective and compact summary of existing relevant works, while urging readers to refer to aforementioned cited sources for exhaustive coverage.

The rest of the paper is then structured as follows: Section II provides a highly concise and minimalist summary of existing works on LCS and LIS. Section III covers preliminaries on LCS and LIS. Section IV formally defines the LSBS problem and details the reduction from LCS. Section V being the most important part of this paper, presents our algorithmic solutions and analysis for LSBS, and explicitly detailed connection to the Hunt-Szymanski algorithm. Section VI discusses applications and directions for future work, as well as experimental validation and trade-offs, while we conclude in Section VII with a wrap up. A link to the actual implementation hosted on GitHub is provided [11].

REFERENCES

- [1] R. A. Wagner and M. J. Fischer, "The String-to-String Correction Problem," *Journal of the ACM (JACM)*, vol. 21, no. 1, pp. 168-173, 1974.
- [2] P. Erdős and G. Szekeres, "A Combinatorial Problem in Geometry," *Composito Mathematica*, no. 2, pp. 463-470, 1935.
- [3] Y. Li, L. Zou, H. Zhang and D. Zhao, "Longest Increasing Subsequence Computation over Streaming Sequences," *IEEE Transactions on Knowledge and Data Engineering*, vol. 30, no. 6, pp. 1036-1049, 2018.
- [4] K. Luo, Y. Xu and H. Zhang, "On-line Scheduling with Monotone Subsequence Constraints," *Theoretical Computer Science*, vol. 768, pp. 13-25, 2019.
- [5] C. L. Mallows, "Patience Sorting," *Bulletin of the Institute of Mathematics and its Applications*, vol. 9, pp. 216-224, 1973.
- [6] J. W. Hunt and T. G. Szymanski, "A Fast Algorithm for Computing Longest Common Subsequences," *Communications of the ACM*, vol. 5, no. 20, pp. 350-353, 1977.
- [7] L. Libralesso, A. Secardin and V. Jost, "Longest Common Subsequence: An Algorithmic Component Analysis," HAL Open Science, 2020.
- [8] L. Bergroth, H. Hakonen and T. Raita, "A survey of Longest Common Subsequence Algorithms," in *Proceedings Seventh International Symposium on String Processing and Information Retrieval. SPIRE 2000*, A Curuna, 2000.
- [9] D. Nath, J. Kurmi and V. Rawat, "A Survey on Longest Common Subsequence," *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, vol. 6, no. 4, pp. 4553-4557, 2018.
- [10] D. Gusfield, *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*, New York: Cambridge University Press, 1997.
- [11] A. Mohammed, "LSBS," GitHub, 2025. [Online]. Available: <https://github.com/awwalm/lsbs-pub>.
- [12] V. I. Levenshtein, "Binary Codes Capable of Correcting Deletions, Insertions, and Reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707-710, 1965.
- [13] D. E. Knuth, *The Art of Computer Programming*, 3 ed., vol. I, Boston: Addison-Wesley, 1997, p. 243.
- [14] S. B. Needleman and C. D. Wunsch, "A General Method Applicable to the Search for Similarities in the Amino Acid Sequence of two Proteins," *Journal of Molecular Biology*, vol. 48, no. 3, pp. 443-453, 1970.
- [15] G. S. Brodal, R. Fagerberg and C. M. Rysgaard, "On Finding Longest Palindromic Subsequences Using Longest Common Subsequences," in *32nd Annual European Symposium on Algorithms (ESA 2024)*, London, 2024.
- [16] M. Kiyomi, H. Ono, Y. Otachi, P. Schweitzer and J. Tarui, "Space-Efficient Algorithms for Longest Increasing Subsequence," in *35th Symposium on Theoretical Aspects of Computer Science (STACS 2018)*, Caen, 2018.
- [17] D. S. Hirschberg, "A Linear Space Algorithm for Computing Maximal Common Subsequences," *Communications of the ACM*, vol. 18, no. 6, pp. 341-343, 1975.
- [18] T. F. Smith and M. S. Waterman, "Identification of Common Molecular Subsequences," *Journal of Molecular Biology*, vol. 147, no. 1, pp. 195-197, 1981.
- [19] S. F. Altschul and D. J. Lipman, "Trees, Stars, and Multiple Biological Sequence Alignment," *SIAM Journal on Applied Mathematics*, vol. 49, no. 1, pp. 197-209, 1989.
- [20] W. J. Masek and M. S. Paterson, "A Faster Algorithm Computing String Edit Distances," *Journal of Computer and System Sciences*, vol. 20, no. 1, pp. 18-31, 1980.
- [21] M. L. Fredman and D. E. Willard, "Surpassing the Information Theoretic Bound with Fusion Trees," *Journal of Computer and System Sciences*, vol. 47, pp. 424-436, 1993.

- [22] Q. Shi, J. F. Jaja and C. W. Mortensen, "Space-Efficient and fast Algorithms for Multidimensional Dominance Reporting and Range Counting," in *Lecture Notes in Computer Science (LNCS)*, vol. 3314, R. Fleischer and G. Trippen, Eds., Berlin, Springer, 2004, pp. 558-568.
- [23] Y. Chen, A. Wan and W. Liu, "A Fast Parallel Algorithm for Finding the Longest Common Subsequence of Multiple Biosequences," *Symposium of Computations in Bioinformatics and Bioscience (SCBB06)*, vol. 7, no. 4, 2006.
- [24] T. Afrin, "The Longest Common Subsequence via Generalized Suffix Trees," WVU, 2015.
- [25] K. Kawanami and N. Fujimoto, "GPU Accelerated Computation of the Longest Common Subsequence," in *Facing the Multicore-Challenge II. Lecture Notes in Computer Science (LNCS)*, vol. 7174, R. Keller, D. Kramer and J. Weiss, Eds., Berlin, Springer, 2012, pp. 84-85.
- [26] B. S. Baker and R. Giancarlo, "Sparse Dynamic Programming for Longest Common Subsequence from Fragments," *Journal of Algorithms*, vol. 42, no. 2, pp. 231-254, 2002.
- [27] D. Eppstein, Z. Galil, R. Giancarlo and G. F. Italiano, "Sparse Dynamic Programming I: Linear Cost Functions," *Journal of the ACM*, vol. 39, no. 3, pp. 519-545, 1992.
- [28] M. T. Goodrich, R. Tamassia and M. H. Goldwasser, *Data Structures and Algorithms in Python*, Hoboken, NJ: John Wiley & Sons, Inc., 2013.
- [29] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed., Cambridge, Massachusetts: MIT Press, 2009.